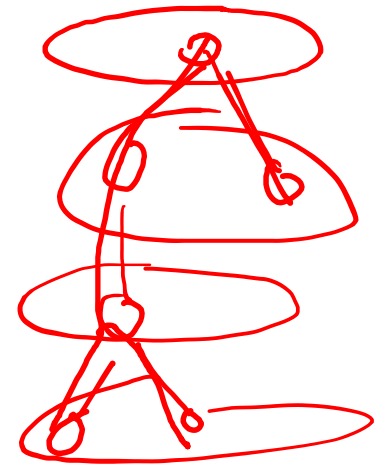
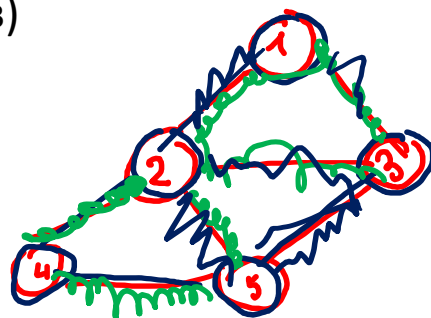


Курс «Комбинаторика и теория графов»

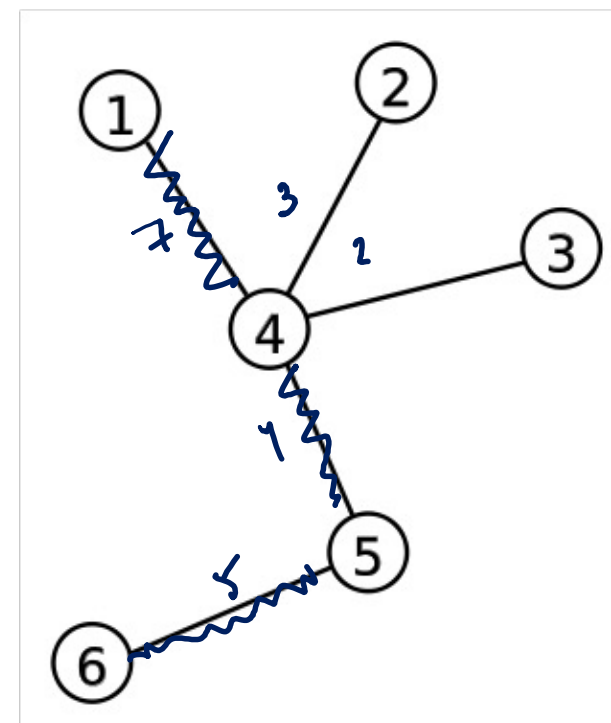
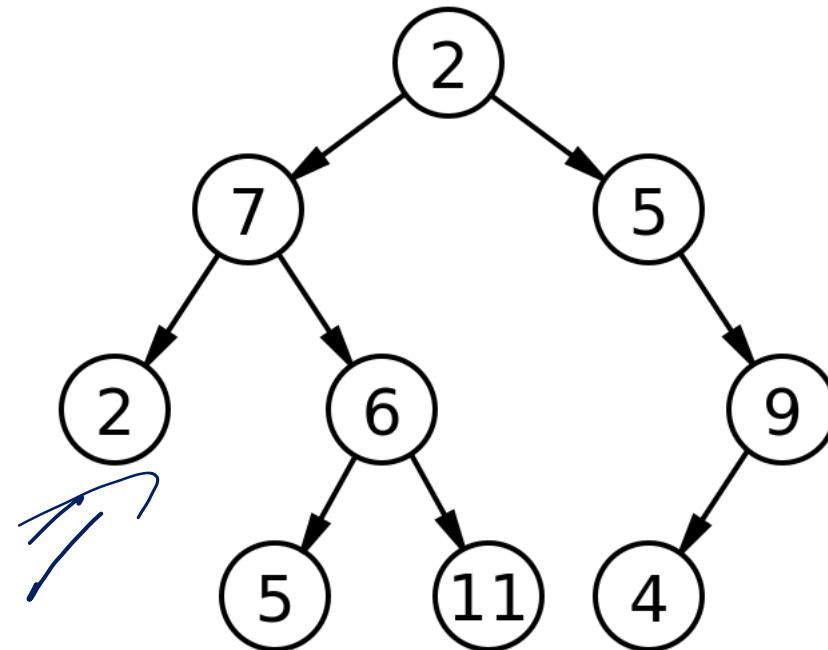
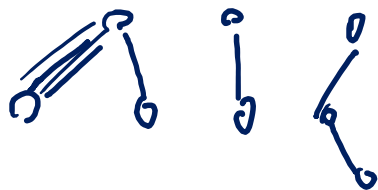
Лекция 4. Теория графов.
Алгоритмы на основе DFS/BFS.



Дерево



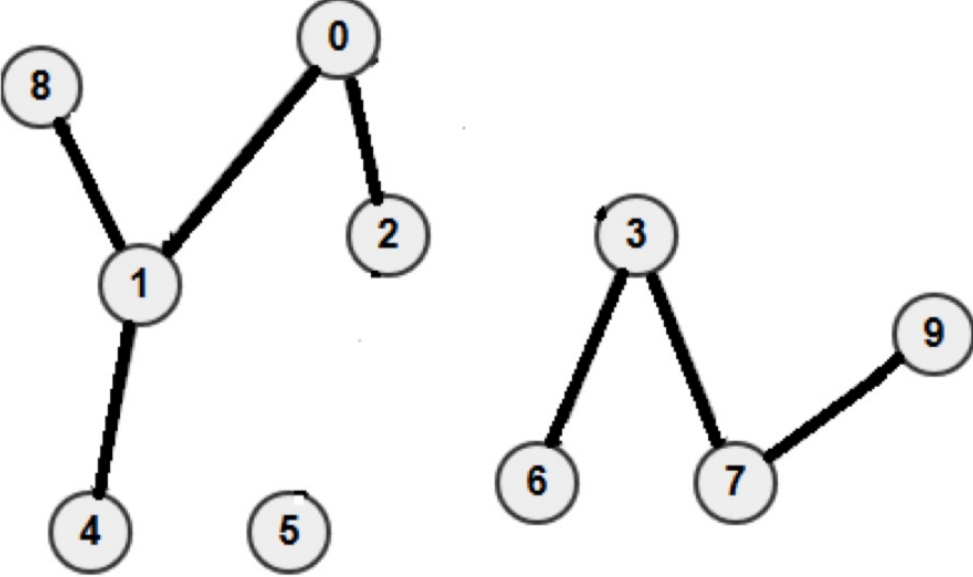
- **Дерево** — это связный ациклический граф.
- **Лес** — множество деревьев.
- **Ориентированное (направленное) дерево** — ациклический орграф, в котором только одна вершина имеет нулевую полустепень захода (в неё не ведут дуги), а все остальные вершины имеют полустепень захода 1 (в них ведёт ровно по одной дуге). Вершина с нулевой полустепенью захода называется корнем дерева, вершины с нулевой полустепенью исхода (из которых не исходит ни одна дуга) называются концевыми вершинами или листьями.



Свойства

- Дерево не имеет кратных рёбер и петель.
- Любое дерево с n вершинами содержит $n - 1$ ребро. Более того, конечный связный граф является деревом тогда и только тогда, когда $B - P = 1$, где B — число вершин, P — число рёбер графа.
- Граф является деревом тогда и только тогда, когда любые две различные его вершины можно соединить единственной простой цепью.
- Любое дерево однозначно определяется расстояниями (длиной наименьшей цепи) между его концевыми (степени 1) вершинами.
- Любое дерево является двудольным графом.

Поиск компонент слабой связности

Несвязный граф G	Вызов DFS из основного текста программы
 {0, 1, 2, 4, 8}, {5}, {3, 6, 7, 9} - КОМПОНЕНТЫ СВЯЗНОСТИ.	<pre data-bbox="1070 471 2497 1213">int n; vector<int> g[MAXN]; bool used[MAXN]; vector<int> components; for (int i = 0; i < n; ++i) used[i] = false; for (int i = 0; i < n; ++i) if (!used[i]) { components.clear(); dfs(i); cout << "Component:"; for (size_t j = 0; j < components.size(); ++j) cout << ' ' << components[j]; cout << endl; } void dfs (int v) { used[v] = true; components.push_back(v); for (auto to : g[v]) { if (!used[to]) dfs(to); } }</pre>

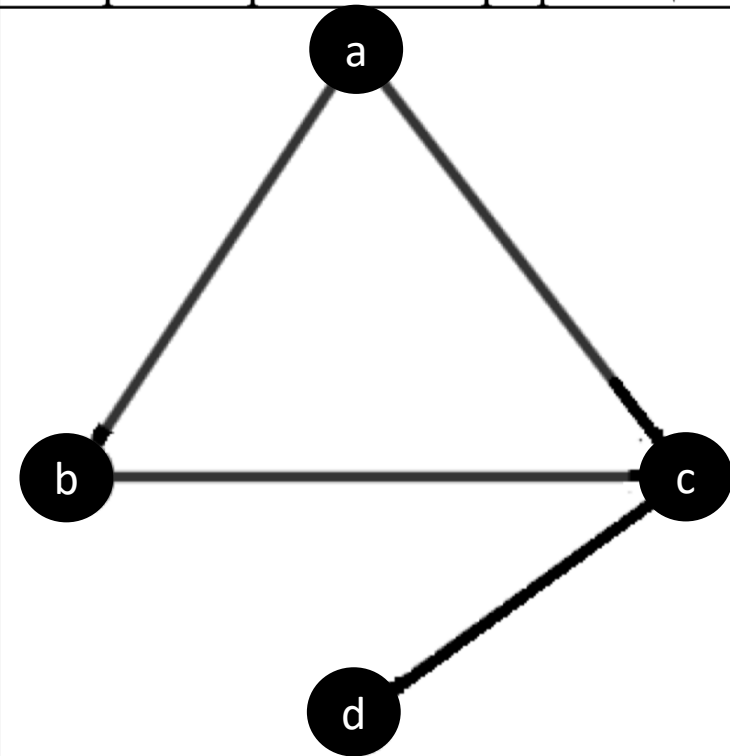
Поиск циклов

Неориентированный граф G. Поиск цикла

```
void dfs (int v) {  
    used[v] = 1;  
    for (auto to : g[v]) {  
        if (used[to] == 0) {  
            p[to] = v;  
            dfs(to);  
        } else  
        if (used[to] == 1 && to != p[v]) {  
            cycle = true;  
        }  
    }  
    used[v] = 2;  
}
```

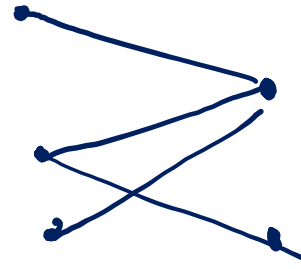
Алгоритм посетит вершину **a**: покрасит ее в цвет 1, запишет предком вершины **b** вершину **a**; посетит вершину **b**: покрасит ее в цвет 1, запишет предком вершины **c** вершину **b**; попытается пройти в вершину **a**, но она уже имеет цвет 1 и при этом не была еще отмечена как предок вершины **c** – значит, алгоритм обнаруживает цикл. Переменная `cycle` становится равной `true`

Неориентированный граф G с циклом

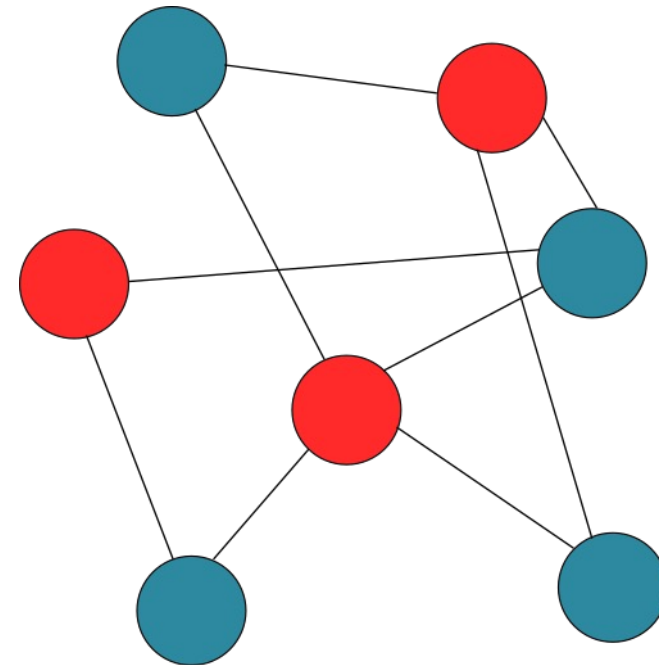
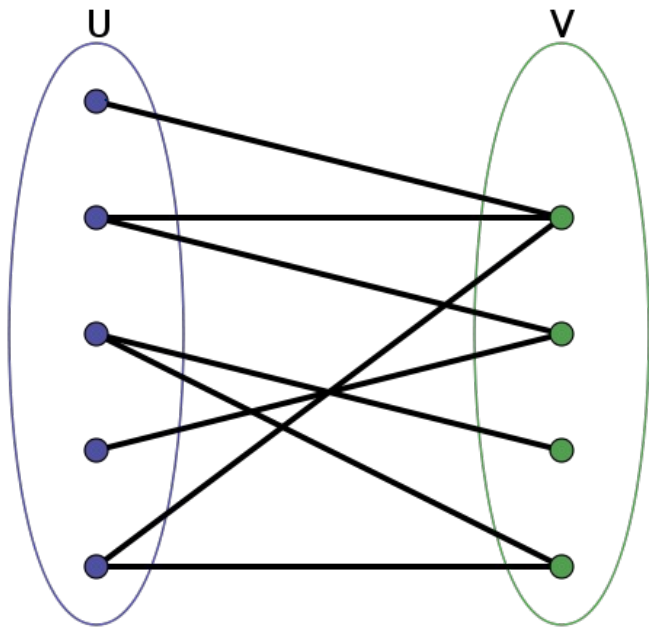


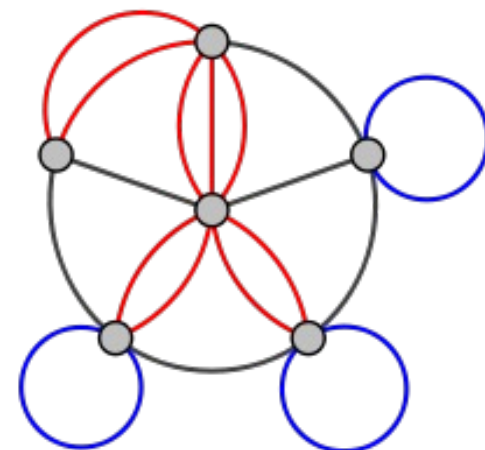
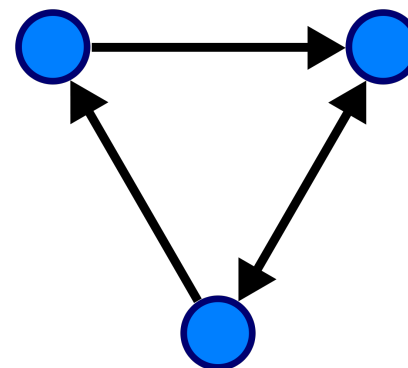
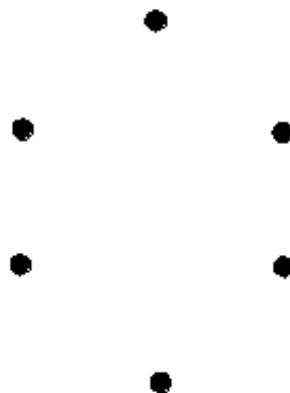
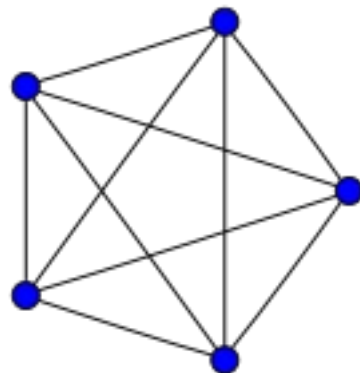
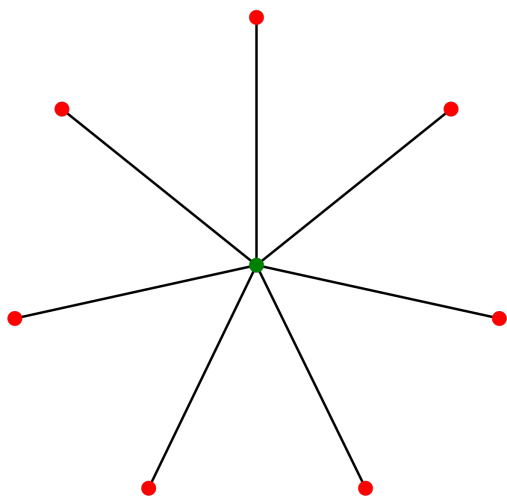
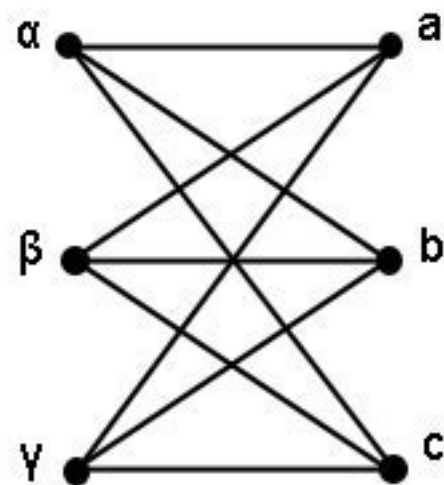
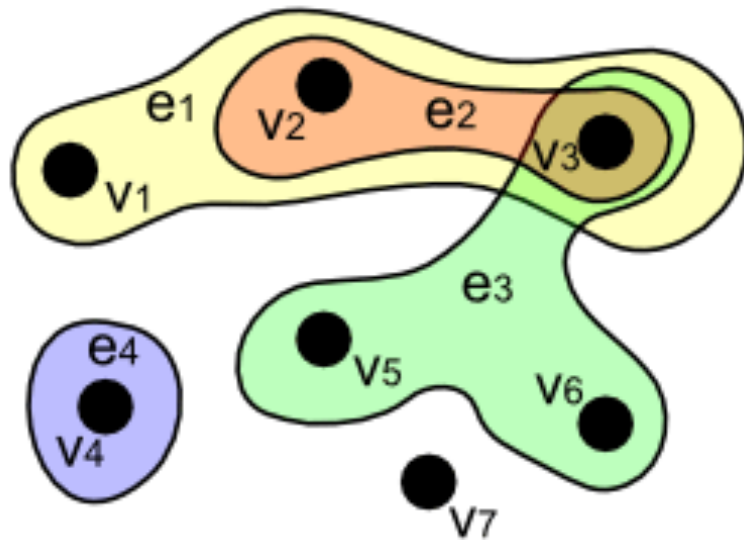
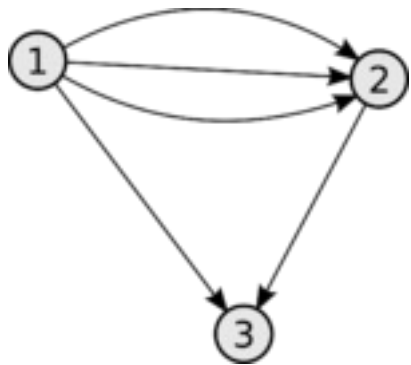
Цикл: $a - b - c$

Двудольный граф



- Двудольный граф - граф, вершины которого можно разбить на два множества так, что каждое ребро соединяет вершины из разных множеств.





Проверка графа на двудольность (DFS)

```
1. void dfs(int v, char c) {    //c - цвет текущей вершины
2.     color[v] = c;
3.     for (int u: graph[v]) {
4.         if (color[u] == 0) {    //непосещённая вершина
5.             dfs(u, invert(c));
6.         } else if (color[u] == c) {
7.             cout << "Graph is not bipartite." << endl;
8.             exit(0);
9.         }
10.    }
11.}

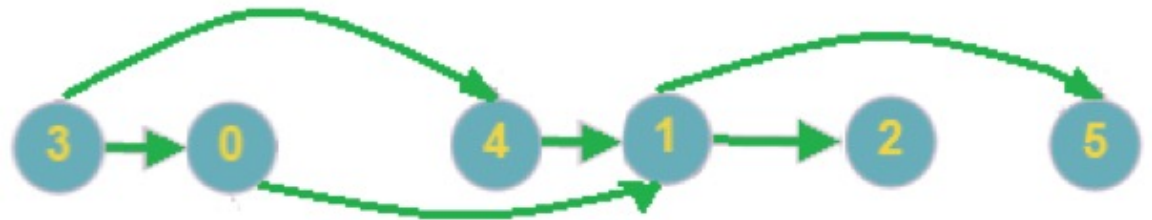
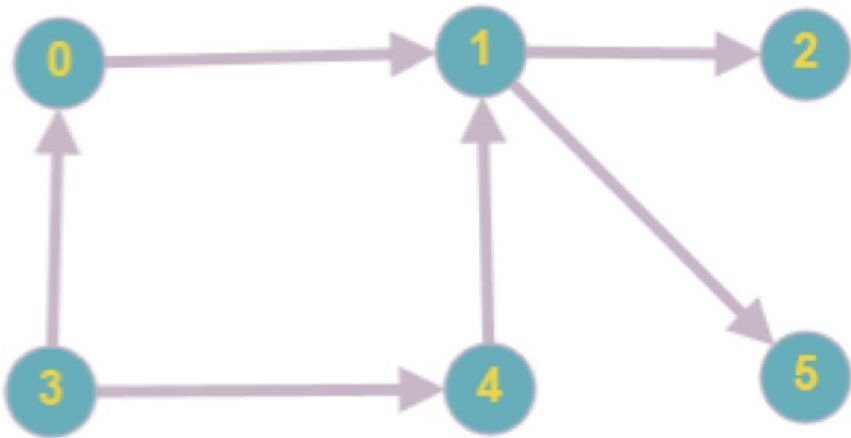
12.int main() {
13.    //Ввод графа...
14.    //Двудольный граф может быть несвязным
15.    for (int i = 0; i < n; i++) {
16.        if (color[i] == 0) {
17.            dfs(i, 1);
18.        }
19.    }
20.    cout << "Graph is bipartite." << endl;
21.}
```


Проверка графа на двудольность (BFS)

- Произведём серию поисков в ширину. Т.е. будем запускать поиск в ширину из каждой непосещённой вершины. Ту вершину, из которой мы начинаем идти, мы помещаем в первую долю. В процессе поиска в ширину, если мы идём в какую-то новую вершину, то мы помещаем её в долю, отличную от доли текущей вершины. Если же мы пытаемся пройти по ребру в вершину, которая уже посещена, то мы проверяем, чтобы эта вершина и текущая вершина находились в разных долях. В противном случае граф двудольным не является. По окончании работы алгоритма мы либо обнаружим, что граф не двудолен, либо найдём разбиение вершин графа на две доли.

Топологическая сортировка графа

- Топологическая сортировка (англ. topological sort) ориентированного ациклического графа $G = (V, E)$ представляет собой упорядочивание вершин таким образом, что для любого ребра $(u, v) \in E(G)$ номер вершины u меньше номера вершины v .



Пример

- Распространённая задача на топологическую сортировку — следующая. Есть n переменных, значения которых нам неизвестны. Известно лишь про некоторые пары переменных, что одна переменная меньше другой. Требуется проверить, не противоречивы ли эти неравенства, и если нет, выдать переменные в порядке их возрастания (если решений несколько — выдать любое). Легко заметить, что это в точности и есть задача о поиске топологической сортировки в графе из n вершин.

Алгоритм на основе DFS (Алгоритм Тарьяна)

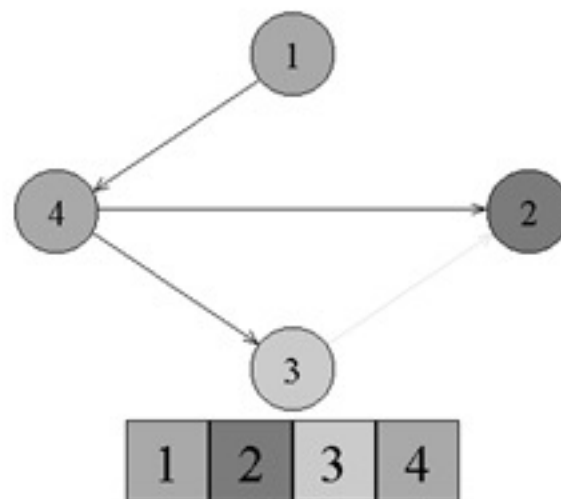
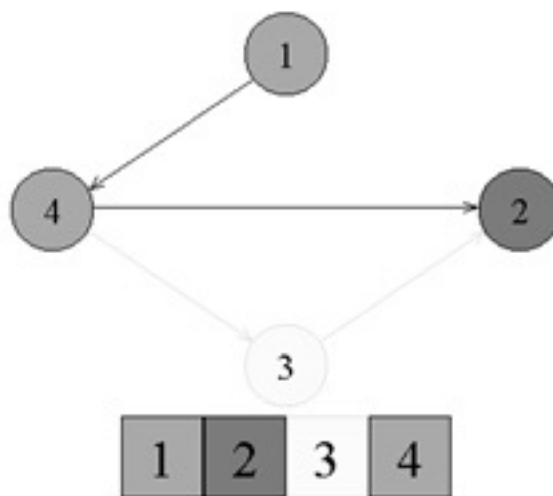
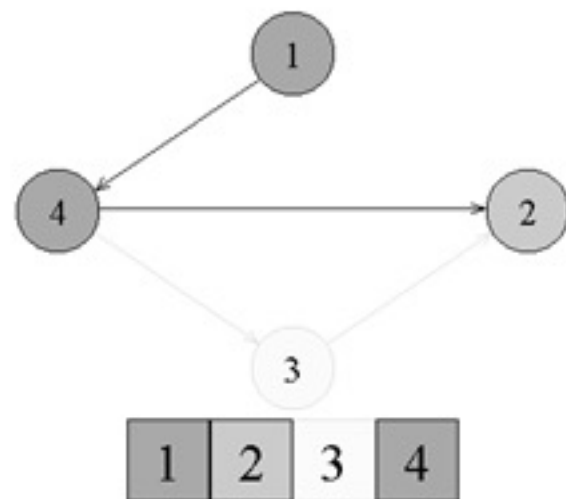
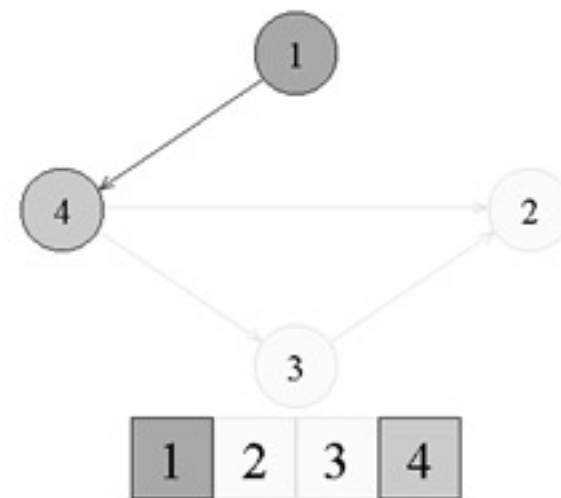
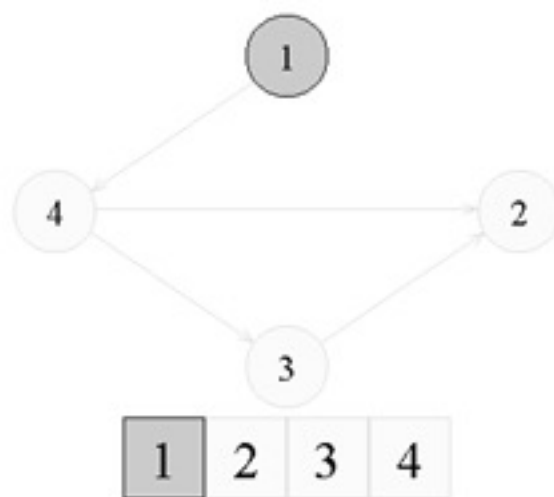
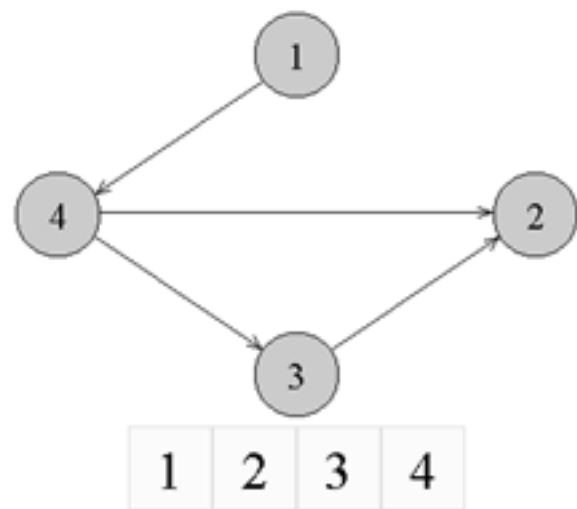
Время работы этого алгоритма соответствует времени работы алгоритма поиска в глубину, то есть равно $O(|V|+|E|)$

- Изначально все вершины белые.
- Для каждой вершины делаем шаг алгоритма.

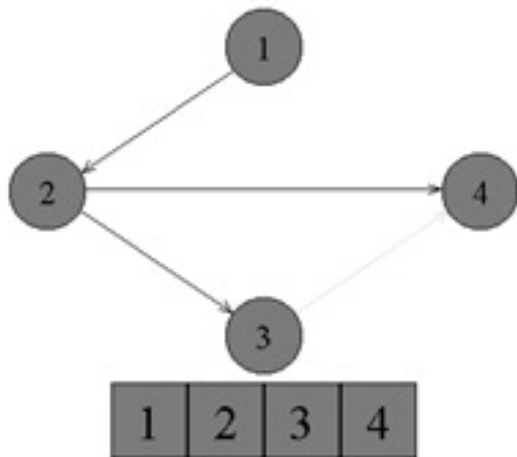
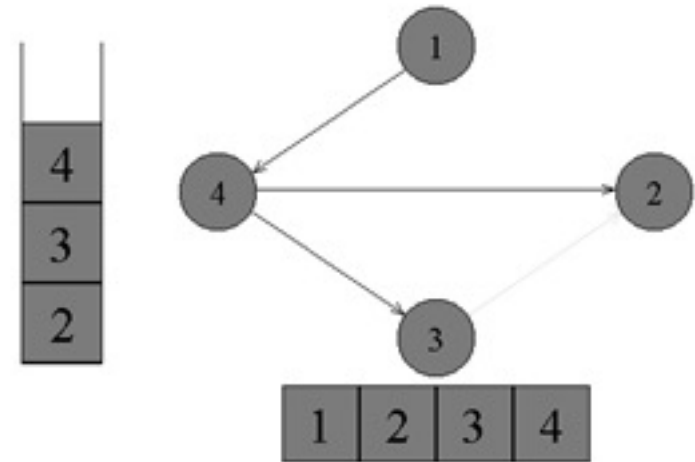
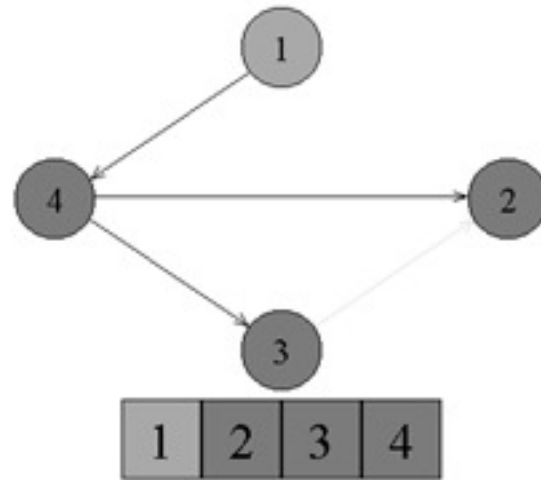
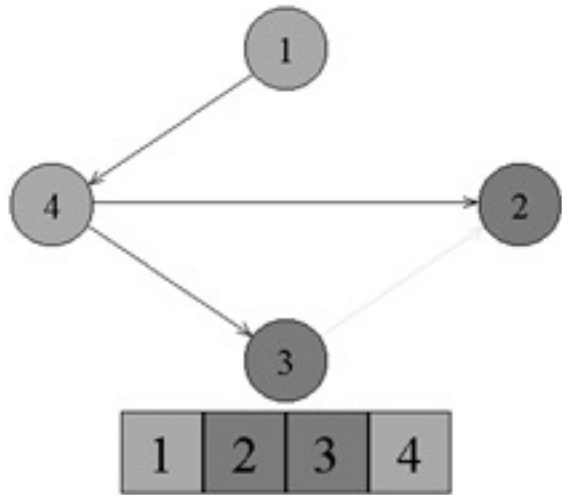
Шаг алгоритма:

- Если вершина чёрная, ничего делать не надо.
- Если вершина серая — найден цикл, топологическая сортировка невозможна.
- Если вершина белая
 - Красим её в серый
 - Применяем шаг алгоритма для всех вершин, в которые можно попасть из текущей
 - Красим вершину в чёрный и помещаем в окончательный список.

Пример



Пример



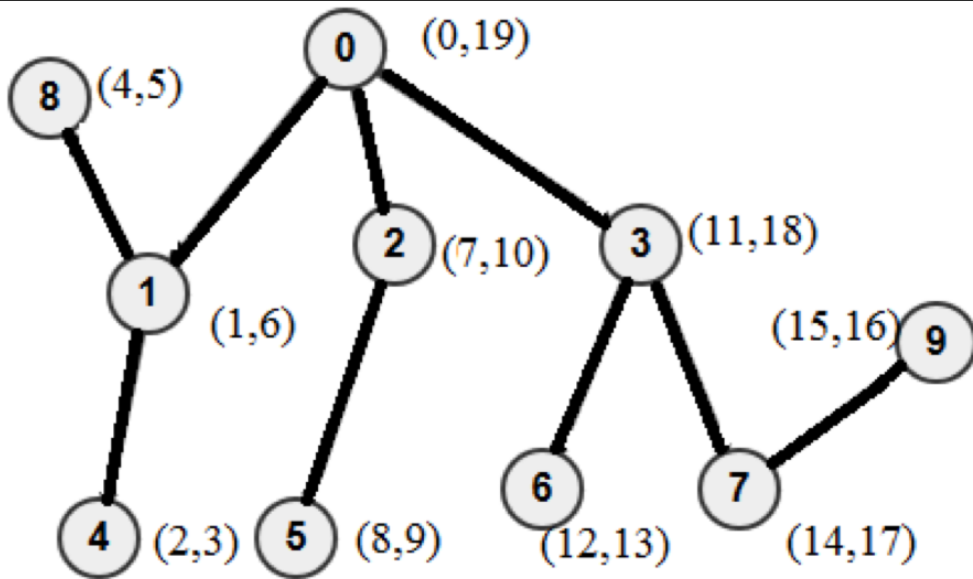
Алгоритм на основе DFS (Алгоритм Тарьяна)

```
1. void dfs(int v)//Функция DFS
2. {
3.     for (size_t i=0; i< g[v].size(); ++i)
4.         if (used[g[v][i]]==0)
5.             dfs(g[v][i]);
6.     used[v]=2; //По окончании обработки
7.     ans.push_back(v); //вершины добавляем ее в вектор ответа
8. }

9. int main() //Основной текст программы
10.{
11./*Считывание данных...*/
12.    for (int i=0;i<n;++i)
13.        if (!used[i])
14.            dfs(i); Запуск от каждой непосещенной вершины
15.    //Вывод вершин ans в обратном порядке
16.    return 0;
17.}
```

Время начала и конца обработки вершины

Неориентированный граф G



Функция обхода графа в глубину DFS с фиксацией времени начала и конца обработки вершин

```
void dfs (int v)
{
    used[v] = 1;
    time_in[v] = time++;
    for (auto i : g[v])
        if (!used[i])
            dfs (*i);
    time_out[v] = time++;
}
```